UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR

POSGRADOS

Maestría en
SOFTWARE

Asignatura:
Patrones de Diseño de Software

Actividad #: 02
Patrones estructurales

Docente:
Mauricio Ortiz Ochoa

Elaborado por (Grupo 02):
Espinoza Adrian
Jara Pablo
Lara Miguel
Torres Carina
Uyaguari Sebastián

Fecha de entrega:
2026/10/19

Repositorio de código:

Patrones estructurales

Introducción

Los patrones estructurales describen cómo combinar clases y objetos para formar estructuras más grandes sin perder flexibilidad (Gamma et al., 1994). En esta actividad se analizan dos escenarios institucionales. El primero exige representar un presupuesto jerárquico en el que las partidas individuales y los grupos anidados deben tratarse de la misma forma; para resolverlo se aplica el patrón Composite. El segundo requiere integrar dispositivos biométricos de un nuevo fabricante, cuya biblioteca tiene una interfaz incompatible, sin alterar el contrato que utiliza el sistema de asistencia de la universidad; para resolverlo se aplica el patrón Adapter. En cada escenario se identifica el problema estructural, se justifica el patrón seleccionado, se presentan los diagramas UML y sus participantes, se describe la implementación en Java y se documenta el uso crítico de la inteligencia artificial.

Escenario 01: Gestión Jerárquica de Presupuestos
Justificación (Gamma, E. et al. (1994) – *Design Patterns (GoF)*)

- **Problema estructural**
  El sistema de la institución necesita modelar una jerarquía de "parte-todo" donde existen elementos primitivos (partidas presupuestarias individuales con código, descripción y valor) y contenedores (grupos presupuestarios que pueden almacenar partidas u otros grupos anidados). El desafío principal radica en que el cliente se ve obligado a realizar diferenciaciones constantes mediante sentencias condicionales para calcular valores totales o mostrar la estructura, lo que rompe el principio de abierto/cerrado (OCP) y aumenta el acoplamiento.

- **Patrón seleccionado:**
  Se selecciona el patrón Composite (Patrón Estructural GoF).

- **Justificación técnica:**

Este patrón permite componer objetos en estructuras de árbol para representar jerarquías parte-todo, permitiendo que los clientes traten de manera uniforme tanto a los objetos individuales (hojas) como a las composiciones complejas (compuestos). Facilita el cálculo recursivo de los valores totales y la incorporación de nuevos niveles de agrupación sin alterar la lógica cliente.

- **Participantes identificados:**
    1. **Componente (IElementoPresupuestario):**
       Declara la interfaz común para los objetos de la composición y define las operaciones (obtenerCodigo(), obtenerDescripcion(), calcularValorTotal(), mostrarEstructura()).

    2. **Hoja (PartidaPresupuestaria):**
       Representa los objetos finales que no tienen hijos. Implementa el comportamiento directo de los datos (código, descripción y valor asignado).

    3. **Compuesto (GrupoPresupuestario):**
       Define el comportamiento para los elementos que tienen hijos, almacena componentes hijos (List<IElementoPresupuestario>) y ejecuta las operaciones de forma recursiva (como la suma acumulada de valores).

    4. **Cliente (MainPresupuestoConcurrente):**
       Manipula los objetos de la estructura exclusivamente a través de la interfaz del componente. Mediante procesarSubarbolParalelo() evalúa subárboles de forma concurrente sin distinguir entre partidas y grupos.

Diagrama (Fowler, M. (2004) – *UML Distilled*.)
Diagrama Original (Estructura base del Composite).

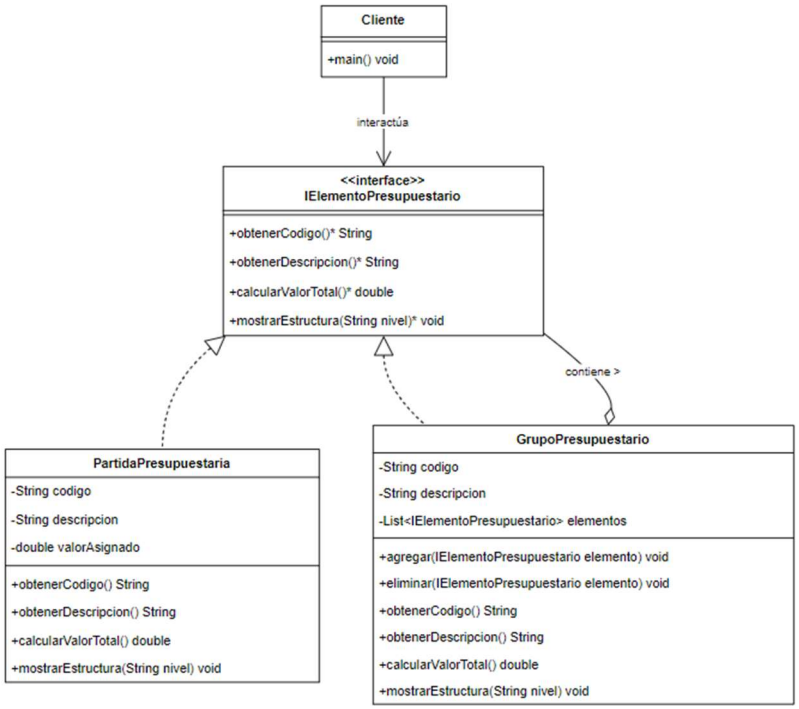


Figura 1: Diagrama de clases inicial de Composite

Diagrama con Cambios Aceptados (Inclusión explícita del Cliente en la jerarquía y procesamiento paralelo).

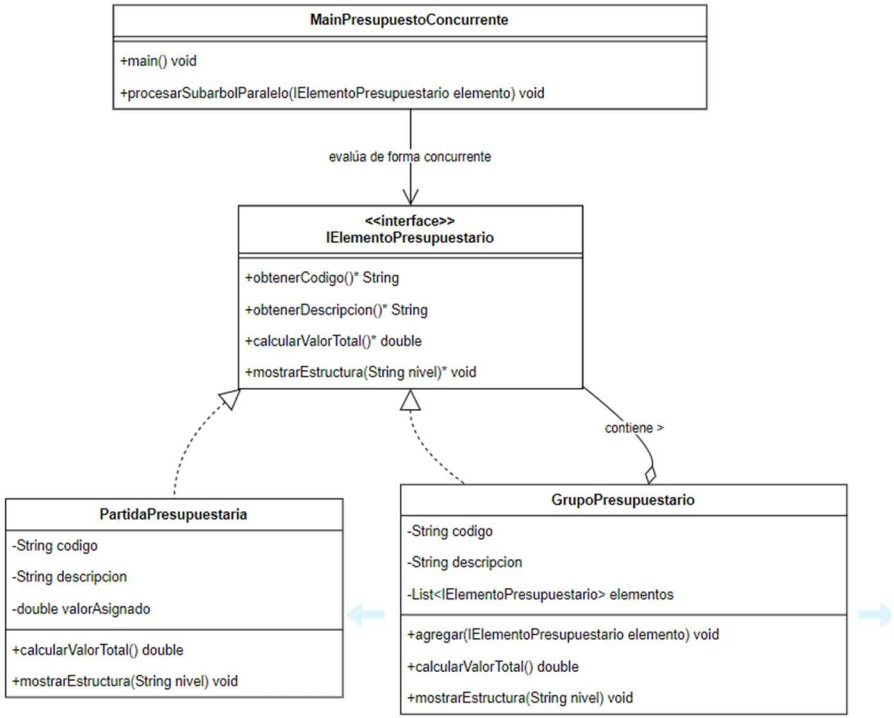


Figura 2: Diagrama Composite final con cambios aceptados

Ejecución de programa java en consola

Ruta de clases java escenario01 en repositorio: https://github.com/carytnas/patrones-diseno-actividad2/tree/main/fuentes/ejercicios/src/escenario01

```
================================================
   SISTEMA DE GESTION PRESUPUESTARIA (COMPOSITE)
================================================
1. Mostrar estructura completa y calculo secuencial
2. Evaluar subarboles de forma PARALELA (Concurrente)
3. Salir
Seleccione una opcion:  1

--- REPORTE SECUENCIAL ---
[+] Grupo: [G-01] Presupuesto Institucional 2026 (Total: $114000.0)
    [+] Grupo: [G-TI] Tecnologias de la Informacion (Total: $106000.0)
        [-] Partida: [P-01] Licenciamiento de software - Valor: $15000.0
        [+] Grupo: [G-INF] Infraestructura (Total: $71000.0)
            [-] Partida: [P-02] Servidores - Valor: $35000.0
            [-] Partida: [P-03] Almacenamiento - Valor: $18000.0
            [+] Grupo: [G-COM] Comunicaciones (Total: $18000.0)
                [-] Partida: [P-04] Switches core - Valor: $12000.0
                [-] Partida: [P-05] Enlaces de internet - Valor: $6000.0
        [+] Grupo: [G-SER] Servicios (Total: $20000.0)
            [-] Partida: [P-06] Soporte tecnico - Valor: $9000.0
            [-] Partida: [P-07] Servicios en la nube - Valor: $11000.0
    [-] Partida: [P-08] Auditoria Externa Sistemas - Valor: $8000.0
Total Consolidado: $114000.0

================================================
   SISTEMA DE GESTION PRESUPUESTARIA (COMPOSITE)
================================================
1. Mostrar estructura completa y calculo secuencial
2. Evaluar subarboles de forma PARALELA (Concurrente)
3. Salir
Seleccione una opcion:  2

--- ANALISIS Y CALCULO PARALELO DE SUBARBOLES ---
 [ForkJoinPool.commonPool-worker-1] [G-TI] Tecnologias de la Informacion -> Subtotal: $106000.0
 [ForkJoinPool.commonPool-worker-2] [P-08] Auditoria Externa Sistemas -> Subtotal: $8000.0
 Total Consolidado: $114000.0
 Tiempo de procesamiento concurrente: 4 ms.

================================================
   SISTEMA DE GESTION PRESUPUESTARIA (COMPOSITE)
================================================
1. Mostrar estructura completa y calculo secuencial
2. Evaluar subarboles de forma PARALELA (Concurrente)
3. Salir
Seleccione una opcion:  3
```

Uso crítico de inteligencia artificial

- Prompt principal:

"Actúa como un arquitecto de software especializado en patrones de diseño GoF y modelado UML. Estoy desarrollando una actividad académica de maestría sobre patrones estructurales.
Aborda la gestión de presupuestos institucionales con niveles jerárquicos (partidas individuales y grupos anidados), he seleccionado el patrón **Composite** y realizado un análisis previo del problema.
Objetivo: Necesito que revises mi selección, me ayudes a estructurar una propuesta de diseño orientada a objetos que evite la diferenciación condicional por parte del cliente y propongas el diagrama de clases correspondiente. Instrucciones:
1. Evalúa si la selección del patrón Composite es adecuada para resolver el problema de estructuras "parte-todo" y justifica técnicamente tu respuesta.
2. Define los participantes estructurales del patrón (Component, Leaf y Composite) aplicados al dominio presupuestario.
3. Propone el diagrama de clases en formato Mermaid manteniendo correspondencia estricta con los requerimientos de cálculo recursivo y uniformidad operativa.
4. Considera la integración de procesamiento concurrente opcional para subárboles de presupuesto.
Consideraciones:
- Utiliza un rigor técnico acorde a un nivel de posgrado/maestría.
- No agregues complejidad ni clases innecesarias fuera del alcance del enunciado.

- **Recomendación aceptada (con justificación):**
La sugerencia de unificar los métodos de impresión de estructura utilizando un parámetro de indentación (String nivel).

  Justificación:
  Permite formatear visualmente la jerarquía en consola de manera limpia y recursiva sin alterar la lógica de negocio de los cálculos.

- **Recomendación modificada o rechazada (con justificación):**
La IA propuso incluir métodos de gestión de hijos (agregar/eliminar) directamente dentro de la interfaz principal del Componente.

  Justificación:
  Se modificó para mantener dichos métodos exclusivamente en la clase GrupoPresupuestario (Composite), siguiendo estrictamente los principios de diseño seguro de GoF para evitar que las hojas expongan operaciones de contenedores que no soportan.

Escenario 02: Integración de Nuevos Dispositivos Biométricos
Justificación (Gamma, E. et al. (1994) – *Design Patterns (GoF)*)

- **Problema estructural:**
  La universidad dispone de un sistema institucional con un contrato de asistencia establecido. La llegada de nuevos dispositivos biométricos de un fabricante diferente introduce una interfaz incompatible basada en colecciones y métodos propios que no coinciden con el estándar operativo interno. Modificar el sistema raíz violaría el desacoplamiento y crearía acoplamiento con proveedores externos.

- **Patrón seleccionado:**
  Se selecciona el patrón Adapter (Patrón Estructural GoF).

- **Justificación técnica:**
  Convierte la interfaz de una clase existente (la nueva biblioteca biométrica del proveedor) en otra interfaz compatible que el cliente espera, permitiendo que clases con interfaces incompatibles colaboren sin modificar sus códigos fuente originales.

- **Participantes identificados:**
  1. **Target (ISistemaAsistencia):**
     Define el dominio específico y la interfaz normalizada que utiliza el sistema de la universidad.
  2. **Adaptee / Adaptado (BibliotecaBiometricaLegacy, CloudBiometricApi):**
     Bibliotecas de terceros, no modificables, con operaciones y tipos propios: getLogsOld(int) devuelve en texto plano todas las marcaciones de un código numérico y fetchUserData(String) recibe una consulta JSON y devuelve un JsonObject con horas en formato ISO-8601.

  3. **Adapter (BiometricoLegacyAdapter, BiometricoNuevoAdapter):**
     Implementan el Target y contienen por composición a su Adaptee (Adapter de objetos). Traducen los parámetros (identificador a código numérico, fecha a formato yyyyMMdd o consulta JSON) y transforman cada respuesta propietaria en un RegistroNormalizado.

  4. **Cliente (MainAsistenciaConcurrente):**
     Interactúa exclusivamente con la interfaz ISistemaAsistencia y con RegistroNormalizado; consulta varios adaptadores en paralelo sin conocer las particularidades de cada fabricante.

Diagrama (Fowler, M. (2004) – *UML Distilled*.)
Diagrama Original (Estructura base del Adapter)

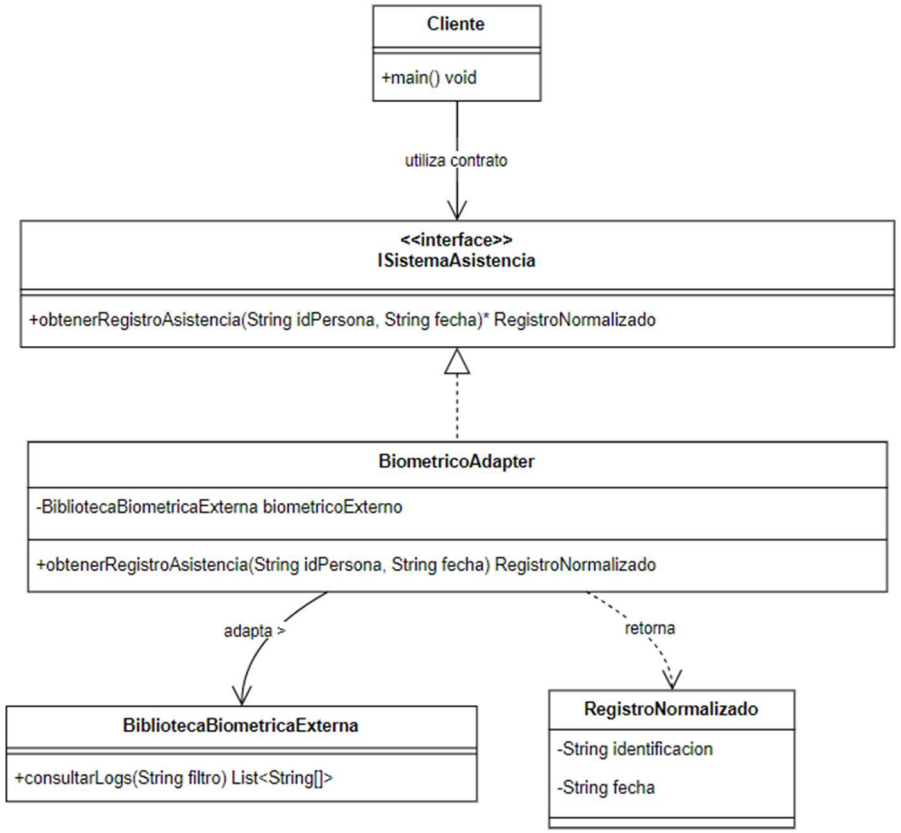


Figura 3: Diagrama Original (Estructura base del Adapter)

Diagrama con Cambios Aceptados (Múltiples adaptadores de proveedores en paralelo y cliente interactivo)

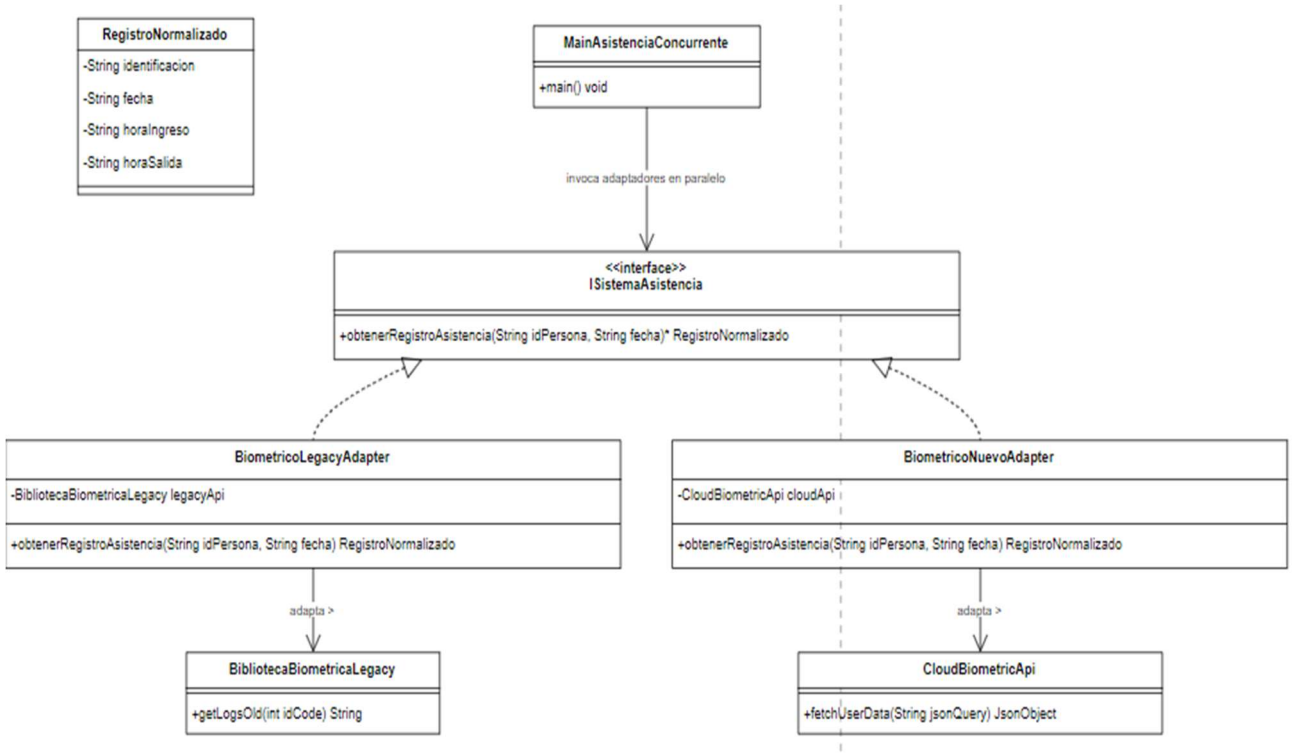


Figura 4: Diagrama (Estructura base del Adapter), con cambios aceptados

Ejecución de proceso java implementado

Ruta de clases java escenario02 en repositorio: https://github.com/carytnas/patrones-diseno-actividad2/tree/main/fuentes/ejercicios/src/escenario02

```
================================================
   SISTEMA DE ASISTENCIA BIOMETRICA (ADAPTER)
================================================
Fecha de consulta: 2026-10-19
1. Consultar asistencia con Dispositivo Legacy (Adaptador 1)
2. Consultar asistencia con Dispositivo Cloud (Adaptador 2)
3. Consultar todos los dispositivos en PARALELO
4. Cambiar fecha de consulta (yyyy-MM-dd)
5. Salir
Seleccione una opcion: 3

--- CONSULTA DE DISPOSITIVOS EN PARALELO ---
[Cloud API]: Consultando API web del nuevo proveedor... {"userId":"COL-1002","day":"20261019"}
[Legacy API]: Consultando dispositivo antiguo...
 Resultados Concurrentes Obtenidos (formato normalizado):
 - ID: COL-1001 | Fecha: 2026-10-19 | Ingreso: 08:10 | Salida: 17:00
 - ID: COL-1002 | Fecha: 2026-10-19 | Ingreso: 07:55 | Salida: 17:15
 Tiempo total de respuesta en paralelo: 5 ms.

================================================
   SISTEMA DE ASISTENCIA BIOMETRICA (ADAPTER)
================================================
Fecha de consulta: 2026-10-19
1. Consultar asistencia con Dispositivo Legacy (Adaptador 1)
2. Consultar asistencia con Dispositivo Cloud (Adaptador 2)
3. Consultar todos los dispositivos en PARALELO
4. Cambiar fecha de consulta (yyyy-MM-dd)
5. Salir
Seleccione una opcion: 4
Nueva fecha (yyyy-MM-dd): 2026-10-20

================================================
   SISTEMA DE ASISTENCIA BIOMETRICA (ADAPTER)
================================================
Fecha de consulta: 2026-10-20
1. Consultar asistencia con Dispositivo Legacy (Adaptador 1)
2. Consultar asistencia con Dispositivo Cloud (Adaptador 2)
3. Consultar todos los dispositivos en PARALELO
4. Cambiar fecha de consulta (yyyy-MM-dd)
5. Salir
Seleccione una opcion: 3

--- CONSULTA DE DISPOSITIVOS EN PARALELO ---
[Cloud API]: Consultando API web del nuevo proveedor... {"userId":"COL-1002","day":"20261020"}
[Legacy API]: Consultando dispositivo antiguo...
 Resultados Concurrentes Obtenidos (formato normalizado):
 - ID: COL-1001 | Fecha: 2026-10-20 | Ingreso: 07:59 | Salida: 17:03
 - ID: COL-1002 | Fecha: 2026-10-20 | Ingreso: --:-- | Salida: --:--
 Tiempo total de respuesta en paralelo: 0 ms.

================================================
   SISTEMA DE ASISTENCIA BIOMETRICA (ADAPTER)
================================================
Fecha de consulta: 2026-10-20
1. Consultar asistencia con Dispositivo Legacy (Adaptador 1)
2. Consultar asistencia con Dispositivo Cloud (Adaptador 2)
3. Consultar todos los dispositivos en PARALELO
4. Cambiar fecha de consulta (yyyy-MM-dd)
5. Salir
Seleccione una opcion: 5
```

Uso crítico de inteligencia artificial

- **Prompt principal:**

  "Actúa como un arquitecto de software experto en integración de sistemas y patrones estructurales GoF. Estoy desarrollando una actividad académica de maestría. Donde una universidad debe integrar dispositivos biométricos de un nuevo fabricante con una interfaz propietaria incompatible sin alterar su contrato institucional de asistencia, he seleccionado el patrón **Adapter**.

  Objetivo:

  Necesito que valides mi enfoque, determines la estrategia de adaptación óptima y elabores el diagrama de clases y la propuesta técnica correspondiente. Instrucciones:

  1. Analiza el problema de incompatibilidad de interfaces heterogéneas y justifica la idoneidad del patrón Adapter.

  2. Identifica con precisión los roles del patrón en este contexto (Target, Adaptee, Adapter y Client).

  3. Diseña una solución orientada a objetos que incluya un modelo de datos normalizado para aislar al sistema institucional de los formatos crudos de los proveedores.
  4. Genera el diagrama de clases en formato Mermaid y modela la capacidad de soportar múltiples adaptadores en paralelo.

  Consideraciones:

  - Mantén estricta correspondencia con los principios SOLID (especialmente OCP).
  - Justifica las decisiones de diseño bajo un enfoque de mantenibilidad empresarial a largo plazo."

- Recomendación aceptada (con justificación): Crear una clase de transferencia de datos RegistroNormalizado para estandarizar el formato de salida devuelto por el adaptador.

  Justificación: Asegura que el sistema cliente no dependa de las estructuras propias de cada proveedor (texto plano y JsonObject).

- Recomendación modificada o rechazada (con justificación): La IA sugirió que el adaptador heredara directamente de la clase del fabricante mediante herencia múltiple (Adapter de clases).

  Justificación: Se rechazó y se modificó por un Adapter de objetos basado en composición, ya que en Java no existe la herencia múltiple de clases y la composición desacopla de forma más robusta las librerías de terceros.

Conclusiones de la Actividad (IEEE / SWEBOK V4.0a (2025) & Asaad & Avksentieva (2025))

La actividad permitió comprobar que los patrones estructurales resuelven problemas concretos de diseño cuando se eligen a partir del contexto. El patrón Composite permitió representar el presupuesto institucional como un árbol en el que partidas y grupos comparten la interfaz IElementoPresupuestario. Gracias a ello, el cliente calcula totales y muestra la estructura sin condicionales, el cálculo recursivo abarca cualquier nivel de anidamiento y es posible incorporar nuevos grupos sin modificar la lógica de consulta, en concordancia con el principio Abierto/Cerrado.

El patrón Adapter, por su parte, permitió integrar dispositivos biométricos con interfaces incompatibles manteniendo estable el contrato ISistemaAsistencia. Cada adaptador concentra las transformaciones propias de su fabricante, como la conversión de identificadores, formatos de fecha y estructuras de respuesta, por lo que integrar un nuevo proveedor solo requiere un nuevo adaptador. En conjunto, ambos patrones reducen el acoplamiento, favorecen la reutilización y preparan a los sistemas institucionales para evolucionar con un menor costo de mantenimiento. Finalmente, el uso de la inteligencia artificial resultó útil para contrastar alternativas, pero exigió validar críticamente sus propuestas: se aceptaron las que mejoraban el diseño y se rechazaron las que contradecían las restricciones del lenguaje o la seguridad del modelo.

Referencias Bibliográficas

- Asaad, J., y Avksentieva, E. (2025). *Assessing the Impact of GoF Design Patterns on Software Engineering Practices*.
- Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3.ª ed.). Addison-Wesley.

- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software* (1.ª ed.). Addison-Wesley.
- IEEE Computer Society. (2025). *Guide to the Software Engineering Body of Knowledge — SWEBOK Guide V4.0a* (Versión 4.0a). IEEE.
- Musch, O. (2023). *Design Patterns with Java: An Introduction* (1.ª ed.). Apress.
- Sarcar, V. (2022). *Java Design Patterns: A Hands-On Experience with Real-World Examples* (3.ª ed.). Apress.
- Shvets, A. (2019). *Sumérgete en los patrones de diseño* (1.ª ed.). Refactoring.Guru.